Department of Computer Engineering & Applications
GLA UNIVERSITY, MATHURA

SEGMENT TREE

Problem Statements (Set 1)

Sanjay Madaan

26

Problem 1: Range Sum Query – Mutable

From LeetCode (LC 307)

---

1. Problem Understanding

You are given an array `nums`.

You need to support two operations efficiently:

1. **Update**
   Change value at index `i`
2. `nums[i] = val`
3. **Query**
   Return sum of elements in range `[l, r]`
4. `sum(nums[l] ... nums[r])`

Why is this tricky?

If you do it naively:

- Update → O(1)
- Query → O(n)

If there are **many queries ($q \approx 10^5$)** → Too slow

---

2. Why Segment Tree?

Segment Tree helps because:

Operation	Time Complexity
Build	O(n)
Update	O(log n)
Query	O(log n)

It divides array into segments and stores **partial sums**

---

3. Core Idea

Each node stores:

```
sum of a segment
```

Example:

```
Array: [1, 3, 5, 7]

Tree:
        16
      /    \
     4      12
    / \    /  \
   1   3  5    7
```

---

4. Approach

Step 1: Build Tree

- Recursively divide array
- Store sum at each node

Step 2: Update

- Go to leaf
- Update value
- Recalculate parent nodes

Step 3: Query

- If segment fully inside → return
- If outside → return 0
- Else → split

---

5. Java Implementation

```
class SegmentTree {
    int[] tree;
    int n;

    public SegmentTree(int[] nums) {
        n = nums.length;
```

```
        tree = new int[4 * n];
        build(nums, 0, 0, n - 1);
    }

    private void build(int[] nums, int node, int start, int end) {
        if (start == end) {
            tree[node] = nums[start];
        } else {
            int mid = (start + end) / 2;
            build(nums, 2 * node + 1, start, mid);
            build(nums, 2 * node + 2, mid + 1, end);
            tree[node] = tree[2 * node + 1] + tree[2 * node + 2];
        }
    }

    public void update(int idx, int val) {
        updateUtil(0, 0, n - 1, idx, val);
    }

    private void updateUtil(int node, int start, int end, int idx, int val) {
        if (start == end) {
            tree[node] = val;
        } else {
            int mid = (start + end) / 2;
            if (idx <= mid)
                updateUtil(2 * node + 1, start, mid, idx, val);
            else
                updateUtil(2 * node + 2, mid + 1, end, idx, val);

            tree[node] = tree[2 * node + 1] + tree[2 * node + 2];
        }
    }

    public int query(int l, int r) {
        return queryUtil(0, 0, n - 1, l, r);
    }

    private int queryUtil(int node, int start, int end, int l, int r) {
        if (r < start || end < l) return 0; // no overlap
        if (l <= start && end <= r) return tree[node]; // total overlap

        int mid = (start + end) / 2;
        return queryUtil(2 * node + 1, start, mid, l, r) +
               queryUtil(2 * node + 2, mid + 1, end, l, r);
    }
}
```

---

6. Dry Run

```
nums = [1, 3, 5]

query(0,2) → 9
update(1,2) → [1,2,5]
query(0,2) → 8
```

7. Key Learning

- Segment Tree is ideal when:
  - Frequent updates + queries
  - Range-based problems
- Think of it as:
  "Divide → Store → Merge"

Problem 2: Range Minimum Query (RMQ)

---

1. Problem Understanding

You are given an array `nums`.

You need to support:

1. **Update**
2. `nums[i] = val`
3. **Query**
4. `return minimum element in range [l, r]`

Example

```
nums = [2, 6, 1, 4, 5]

query(1,3) → min(6,1,4) = 1
update(2,7) → nums = [2,6,7,4,5]
query(1,3) → min(6,7,4) = 4
```

Why not brute force?

- Query → O(n)
- Too slow for large `q`

---

2. Why Segment Tree?

Same idea as before, but now:

Instead of storing **sum**, we store **minimum**

3. Key Insight

Segment tree is **generic**:

Problem Type	Stored Value
Sum	sum
Minimum	min
Maximum	max
GCD	gcd

Only the **merge operation changes**

---

4. Core Logic Change

Instead of:

```
tree[node] = left + right;
```

We use:

```
tree[node] = Math.min(left, right);
```

---

5. Tree Representation

```
Array: [2, 6, 1, 4]

Tree:
        1
      /   \
     2     1
    / \   / \
   2   6 1   4
```

---

6. Java Implementation

```
class SegmentTreeMin {
    int[] tree;
    int n;

    public SegmentTreeMin(int[] nums) {
        n = nums.length;
        tree = new int[4 * n];
        build(nums, 0, 0, n - 1);
    }

    private void build(int[] nums, int node, int start, int end) {
        if (start == end) {
            tree[node] = nums[start];
        } else {
```

```
            int mid = (start + end) / 2;
            build(nums, 2 * node + 1, start, mid);
            build(nums, 2 * node + 2, mid + 1, end);
            tree[node] = Math.min(tree[2 * node + 1], tree[2 * node + 2]);
        }
    }

    public void update(int idx, int val) {
        updateUtil(0, 0, n - 1, idx, val);
    }

    private void updateUtil(int node, int start, int end, int idx, int val) {
        if (start == end) {
            tree[node] = val;
        } else {
            int mid = (start + end) / 2;
            if (idx <= mid)
                updateUtil(2 * node + 1, start, mid, idx, val);
            else
                updateUtil(2 * node + 2, mid + 1, end, idx, val);

            tree[node] = Math.min(tree[2 * node + 1], tree[2 * node + 2]);
        }
    }

    public int query(int l, int r) {
        return queryUtil(0, 0, n - 1, l, r);
    }

    private int queryUtil(int node, int start, int end, int l, int r) {
        if (r < start || end < l)
            return Integer.MAX_VALUE; // IMPORTANT

        if (l <= start && end <= r)
            return tree[node];

        int mid = (start + end) / 2;
        return Math.min(
            queryUtil(2 * node + 1, start, mid, l, r),
            queryUtil(2 * node + 2, mid + 1, end, l, r)
        );
    }
}
```

---

7. Important Difference from Sum Query

Case	Sum Query	Min Query
No overlap	return 0	return ∞
Merge	+	min()

8. Dry Run

```
nums = [2, 6, 1, 4]

query(1,3) → 1
update(2,7) → [2,6,7,4]
query(1,3) → 4
```

---

9. Key Learning

- Segment Tree is **not a data structure**
  It is a **framework**

You just change:

- What you store
- How you merge

Problem 3: Range Maximum Query (RMQ – Max)

---

1. Problem Understanding

You are given an array `nums`.

You need to support:

1. **Update**
2. `nums[i] = val`
3. **Query**
4. `return maximum element in range [l, r]`

Example

```
nums = [2, 6, 1, 4, 5]

query(1,3) → max(6,1,4) = 6
update(2,7) → nums = [2,6,7,4,5]
query(1,3) → max(6,7,4) = 7
```

Why this problem matters?

It reinforces a **very important realization**:

Segment Tree logic remains SAME
Only the **merge operation changes**

---

2. Why Segment Tree?

Same reasoning:

Operation	Complexity
Build	O(n)
Update	O(log n)
Query	O(log n)

---

3. Core Idea

Each node stores:

```
maximum value of that segment
```

Tree Example

```
Array: [2, 6, 1, 4]

Tree:
        6
      /   \
     6     4
    / \   / \
   2   6 1   4
```

---

4. Key Logic Change

Instead of:

```
tree[node] = left + right;     // Sum
tree[node] = Math.min(left, right);  // Min
```

Now:

```
tree[node] = Math.max(left, right);
```

---

5. Java Implementation

```
class SegmentTreeMax {
    int[] tree;
    int n;

    public SegmentTreeMax(int[] nums) {
        n = nums.length;
        tree = new int[4 * n];
        build(nums, 0, 0, n - 1);
    }

    private void build(int[] nums, int node, int start, int end) {
        if (start == end) {
            tree[node] = nums[start];
        } else {
            int mid = (start + end) / 2;
            build(nums, 2 * node + 1, start, mid);
            build(nums, 2 * node + 2, mid + 1, end);
            tree[node] = Math.max(tree[2 * node + 1], tree[2 * node + 2]);
```

```
        }
    }

    public void update(int idx, int val) {
        updateUtil(0, 0, n - 1, idx, val);
    }

    private void updateUtil(int node, int start, int end, int idx, int val) {
        if (start == end) {
            tree[node] = val;
        } else {
            int mid = (start + end) / 2;
            if (idx <= mid)
                updateUtil(2 * node + 1, start, mid, idx, val);
            else
                updateUtil(2 * node + 2, mid + 1, end, idx, val);

            tree[node] = Math.max(tree[2 * node + 1], tree[2 * node + 2]);
        }
    }

    public int query(int l, int r) {
        return queryUtil(0, 0, n - 1, l, r);
    }

    private int queryUtil(int node, int start, int end, int l, int r) {
        if (r < start || end < l)
            return Integer.MIN_VALUE; // IMPORTANT

        if (l <= start && end <= r)
            return tree[node];

        int mid = (start + end) / 2;
        return Math.max(
            queryUtil(2 * node + 1, start, mid, l, r),
            queryUtil(2 * node + 2, mid + 1, end, l, r)
        );
    }
}
```

---

6. Critical Difference (Interview Trap)

Case	Min Query	Max Query
No overlap	+∞	-∞
Merge	min()	max()

Many students forget:

```
return Integer.MIN_VALUE;
```

---

7. Dry Run

```
nums = [2, 6, 1, 4]

query(1,3) → 6
update(2,7) → [2,6,7,4]
query(1,3) → 7
```

---

8. Key Learning

At this point, you should clearly see:

Segment Tree = **Reusable template**

You only change:

- Merge operation
- Identity value (0, ∞, -∞)

Problem 4: Count Even Numbers in Range

---

1. Problem Understanding

You are given an array `nums`.

You need to support:

1. **Update**
2. `nums[i] = val`
3. **Query**
4. `count how many EVEN numbers exist in range [l, r]`

Example

```
nums = [1, 2, 3, 4, 5]

query(0,4) → 2 (2,4)
update(2,6) → nums = [1,2,6,4,5]
query(0,4) → 3 (2,6,4)
```

Why is this different?

Till now:

- We stored values (sum/min/max)

Now we store a **property (even or not)**

---

2. Why Segment Tree?

Same logic:

- Frequent updates
- Frequent queries

Brute force → O(n) per query □
Segment Tree → O(log n) □

---

3. Key Idea

Convert array into:

```
Even → 1
Odd  → 0
```

So:

```
[1,2,3,4,5] → [0,1,0,1,0]
```

Now problem becomes:

"Find SUM in range"

---

4. Segment Tree Meaning

Each node stores:

```
count of even numbers in that segment
```

Example Tree

```
nums = [1,2,3,4]

converted = [0,1,0,1]

Tree:
        2
      /   \
     1     1
    / \   / \
   0   1 0   1
```

---

5. Approach

Build:

```
tree[node] = (nums[i] % 2 == 0) ? 1 : 0;
```

Merge:

```
tree[node] = left + right;
```

Update:

Recalculate parity (even/odd)

---

6. Java Implementation

```
class SegmentTreeEvenCount {
    int[] tree;
    int n;

    public SegmentTreeEvenCount(int[] nums) {
        n = nums.length;
        tree = new int[4 * n];
        build(nums, 0, 0, n - 1);
    }

    private void build(int[] nums, int node, int start, int end) {
        if (start == end) {
            tree[node] = (nums[start] % 2 == 0) ? 1 : 0;
        } else {
            int mid = (start + end) / 2;
            build(nums, 2 * node + 1, start, mid);
            build(nums, 2 * node + 2, mid + 1, end);
            tree[node] = tree[2 * node + 1] + tree[2 * node + 2];
        }
    }

    public void update(int idx, int val) {
        updateUtil(0, 0, n - 1, idx, val);
    }

    private void updateUtil(int node, int start, int end, int idx, int val) {
        if (start == end) {
            tree[node] = (val % 2 == 0) ? 1 : 0;
        } else {
            int mid = (start + end) / 2;
            if (idx <= mid)
                updateUtil(2 * node + 1, start, mid, idx, val);
            else
                updateUtil(2 * node + 2, mid + 1, end, idx, val);

            tree[node] = tree[2 * node + 1] + tree[2 * node + 2];
        }
    }

    public int query(int l, int r) {
        return queryUtil(0, 0, n - 1, l, r);
    }

    private int queryUtil(int node, int start, int end, int l, int r) {
        if (r < start || end < l)
            return 0;
```

```
        if (l <= start && end <= r)
            return tree[node];

        int mid = (start + end) / 2;
        return queryUtil(2 * node + 1, start, mid, l, r) +
               queryUtil(2 * node + 2, mid + 1, end, l, r);
    }
}
```

---

7. Dry Run

```
nums = [1,2,3,4]

query(0,3) → 2
update(2,6) → [1,2,6,4]
query(0,3) → 3
```

---

8. Key Learning (Very Important)

This problem teaches:

Segment Tree is NOT about values
It is about **what you want to store**

General Pattern

Problem	Store
Sum	value
Min	value
Max	value
Even count	condition
Prime count	condition
Negative count	condition

Problem 5: Range GCD Query

---

1. Problem Understanding

You are given an array `nums`.

You need to support:

1. **Update**
2. `nums[i] = val`
3. **Query**
4. `find GCD of elements in range [l, r]`

Example

```
nums = [2, 4, 6, 8]

query(0,2) → gcd(2,4,6) = 2
update(1,3) → [2,3,6,8]
query(0,2) → gcd(2,3,6) = 1
```

Why is this interesting?

Because:

GCD is **not additive** like sum
But it is **associative**

---

2. Why Segment Tree?

We rely on this property:

```
gcd(a, b, c) = gcd(gcd(a, b), c)
```

This makes it perfect for segment tree

---

3. Key Idea

Each node stores:

```
GCD of that segment
```

Example Tree

```
nums = [2, 4, 6, 8]

Tree:
        2
      /   \
     2     2
    / \   / \
   2   4 6   8
```

---

4. Important Observations

Case	Return Value
No overlap	0
Why?	gcd(x, 0) = x

This is the **identity element**

---

5. Merge Operation

```
tree[node] = gcd(left, right);
```

---

6. Java Implementation

```
class SegmentTreeGCD {
    int[] tree;
    int n;

    public SegmentTreeGCD(int[] nums) {
        n = nums.length;
        tree = new int[4 * n];
        build(nums, 0, 0, n - 1);
    }

    private int gcd(int a, int b) {
        if (b == 0) return a;
        return gcd(b, a % b);
    }
```

```
    private void build(int[] nums, int node, int start, int end) {
        if (start == end) {
            tree[node] = nums[start];
        } else {
            int mid = (start + end) / 2;
            build(nums, 2 * node + 1, start, mid);
            build(nums, 2 * node + 2, mid + 1, end);
            tree[node] = gcd(tree[2 * node + 1], tree[2 * node + 2]);
        }
    }

    public void update(int idx, int val) {
        updateUtil(0, 0, n - 1, idx, val);
    }

    private void updateUtil(int node, int start, int end, int idx, int val) {
        if (start == end) {
            tree[node] = val;
        } else {
            int mid = (start + end) / 2;
            if (idx <= mid)
                updateUtil(2 * node + 1, start, mid, idx, val);
            else
                updateUtil(2 * node + 2, mid + 1, end, idx, val);

            tree[node] = gcd(tree[2 * node + 1], tree[2 * node + 2]);
        }
    }

    public int query(int l, int r) {
        return queryUtil(0, 0, n - 1, l, r);
    }

    private int queryUtil(int node, int start, int end, int l, int r) {
        if (r < start || end < l)
            return 0; // identity for gcd

        if (l <= start && end <= r)
            return tree[node];

        int mid = (start + end) / 2;
        return gcd(
            queryUtil(2 * node + 1, start, mid, l, r),
            queryUtil(2 * node + 2, mid + 1, end, l, r)
        );
    }
}
```

---

7. Dry Run

```
nums = [2,4,6]

query(0,2) → 2
update(1,3) → [2,3,6]
```

```
query(0,2) → 1
```

---

8. Key Learning (Very Important Concept)

Segment tree works when operation is:

Associative

Operation	Works?
Sum	✔
Min/Max	✔
GCD	✔
XOR	✔
Median	□ (not associative)

---

9. Deep Insight

To use segment tree, operation must have:

1. Associativity
2. Identity element